

HƯỚNG DẪN THỰC HÀNH – ESHOP MICROSERVICES

Nền tảng: .NET 8 | Docker | PostgreSQL | MSSQL | SQLite | Redis | RabbitMQ | gRPC | YARP | Refit

1. TỔNG QUAN DỰ ÁN

1.1. Mô tả

EShopMicroservices là ứng dụng thương mại điện tử (e-commerce) mẫu, được xây dựng trên nền tảng .NET 8 theo kiến trúc Microservices. Đây là một dự án học thuật giúp sinh viên hiểu và áp dụng các pattern kiến trúc hiện đại trong phát triển phần mềm doanh nghiệp.

Dự án bao gồm các thành phần chính: Catalog Service (quản lý sản phẩm), Basket Service (giỏ hàng và checkout), Discount gRPC Service (mã giảm giá), Ordering Service (xử lý đơn hàng theo Clean Architecture + DDD), YARP API Gateway (điều phối request) và Shopping.Web (giao diện Razor Pages).

1.2. Mục tiêu học tập

Dự án giúp sinh viên hiểu và thực hành:

- Microservices Architecture: Tách biệt từng domain thành service độc lập, mỗi service có database riêng
- CQRS Pattern: Phân tách Command (ghi) và Query (đọc) với MediatR
- Domain-Driven Design (DDD): Áp dụng trong Ordering service với Aggregate, Entity, Value Object, Domain Event
- Clean Architecture: Ordering service được tổ chức thành 4 tầng: Domain, Application, Infrastructure, API
- Vertical Slice Architecture: Catalog và Basket tổ chức code theo feature, không theo tầng
- Decorator Pattern: CachedBasketRepository bọc thêm cache qua Redis mà không sửa BasketRepository gốc
- gRPC Communication: Giao tiếp đồng bộ giữa Basket và Discount Service
- Async Messaging: RabbitMQ qua MassTransit cho luồng Checkout -> Create Order
- API Gateway Pattern: YARP làm reverse proxy, điều phối request tới các services
- Refit HTTP Client: Shopping.Web gọi API qua interface-based HTTP client

1.3. Công nghệ sử dụng

Thành phần	Công nghệ	Ghi chú
Backend Framework	ASP.NET Core 8 (Minimal API)	Catalog, Basket, Ordering API
Database (Catalog)	PostgreSQL + Marten	Document DB, tự động schema migration
Database (Basket)	PostgreSQL + Marten	Lưu ShoppingCart theo UserName
Database (Discount)	SQLite + EF Core	Nhẹ, chủ yếu dùng cho development
Database (Ordering)	SQL Server (MSSQL) + EF Core	Relational DB, hỗ trợ DDD complex mapping
Cache	Redis (StackExchange.Redis)	Distributed cache cho Basket service
Message Broker	RabbitMQ qua MassTransit	Async event giữa Basket và Ordering
gRPC	.NET gRPC (Grpc.AspNetCore)	Sync RPC từ Basket đến Discount service
API Gateway	YARP (Yet Another Reverse Proxy)	Routing, Rate Limiting, Load Balancing
CQRS/Mediator	MediatR	Xử lý Command và Query
Routing (Minimal API)	Carter	Module-based endpoint registration
Validation	FluentValidation	Pipeline behavior với MediatR
Object Mapping	Mapster	Nhanh hơn AutoMapper, zero config cơ bản
HTTP Client	Refit	Interface-based REST client cho Shopping.Web
Web UI	ASP.NET Razor Pages	Shopping.Web frontend
Containerization	Docker / Docker Compose	Toàn bộ infrastructure + services
Health Check	AspNetCore.HealthChecks	Kiểm tra trạng thái service + DB

2. KIẾN TRÚC HỆ THỐNG

2.1. Sơ đồ tổng quan

Hệ thống bao gồm 6 thành phần chính chạy trong Docker:

```
+-----+
| Shopping.Web | http://localhost:6005
| (Razor Pages) |
+-----+-----+
| (Refit HTTP)
+-----v-----+
| YARP API Gateway | http://localhost:6004
| (Reverse Proxy) |
+---+-----+-----+
| | |
+---v---+ +---v---+ +---v---+
|Cat. | |Bas. | |Ord. |
|API | |API | |API |
|:6000| |:6001| |:6003|
+---+---+ +---+---+ +---+---+
| | |
+---+---+ Redis gRPC SQL
|PgSQL| | Dis.:6002 Svr
|CatDB| PgSQL (SQLite)
+-----+ BasDB
+-----+
| RabbitMQ | :5672 / :15672
+-----+
```

2.2. Cấu trúc thư mục dự án

```

src/
+-- ApiGateways/
|   +-- YarpApiGateway/           # YARP Reverse Proxy
+-- BuildingBlocks/
|   +-- BuildingBlocks/           # CQRS interfaces, Behaviors, Exceptions,
    Pagination
|   +-- BuildingBlocks.Messaging/ # Integration Events, MassTransit config
+-- Services/
|   +-- Catalog/
|       |   +-- Catalog.API/       # Vertical Slice (Product CRUD)
|   +-- Basket/
|       |   +-- Basket.API/        # Vertical Slice (Cart + Checkout)
|   +-- Discount/
|       |   +-- Discount.Grpc/     # gRPC service (Coupon CRUD)
|   +-- Ordering/
|       |   +-- Ordering.Domain/   # DDD Models, Events, Value Objects
|       |   +-- Ordering.Application/ # CQRS Commands/Queries, Event Handlers
|       |   +-- Ordering.Infrastructure/ # EF Core, Interceptors, Migrations
|       |   +-- Ordering.API/      # Minimal API Endpoints
+-- WebApps/
|   +-- Shopping.Web/             # Razor Pages UI
+-- docker-compose.yml
+-- docker-compose.override.yml

```

2.3. Port mapping

Service	HTTP Port	HTTPS Port	Mô tả
catalog.api	6000	6060	Catalog Microservice
basket.api	6001	6061	Basket Microservice
discount.grpc	6002	6062	Discount gRPC Service
ordering.api	6003	6063	Ordering Microservice
yarpapigateway	6004	6064	YARP API Gateway
shopping.web	6005	6065	Razor Pages Web UI
catalogdb (PostgreSQL)	5432	-	Catalog database
basketdb (PostgreSQL)	5433	-	Basket database
distributedcache (Redis)	6379	-	Redis cache
orderdb (SQL Server)	1433	-	Ordering database
messagebroker (RabbitMQ)	5672 / 15672	-	AMQP / Management UI

3. BUILDING BLOCKS - NỀN TẢNG DÙNG CHUNG

BuildingBlocks là thư viện dùng chung chứa các abstraction, behavior và utility được tất cả các services sử dụng lại. Có 2 package: BuildingBlocks (CQRS, Exceptions, Behaviors) và BuildingBlocks.Messaging (Events, MassTransit).

3.1. CQRS Interfaces

Bộ 4 interface giúp type-safe phân tách Command và Query:

```
// ICommand - đánh dấu Command (thay đổi trạng thái)
public interface ICommand : ICommand<Unit> { }
public interface ICommand<out TResponse> : IRequest<TResponse> { }

// ICommandHandler - xử lý Command
public interface ICommandHandler<in TCommand, TResponse>
    : IRequestHandler<TCommand, TResponse>
    where TCommand : ICommand<TResponse>
    where TResponse : notnull { }

// IQuery - đánh dấu Query (chỉ đọc, không thay đổi)
public interface IQuery<out TResponse> : IRequest<TResponse>
    where TResponse : notnull { }

// IQueryHandler - xử lý Query
public interface IQueryHandler<in TQuery, TResponse>
    : IRequestHandler<TQuery, TResponse>
    where TQuery : IQuery<TResponse>
    where TResponse : notnull { }
```

3.2. Pipeline Behaviors (MediatR)

2 behavior tự động chèn vào pipeline của mỗi request:

ValidationBehavior

Chạy trước Handler, kết hợp tất cả IValidator<TRequest> đã đăng ký và throw ValidationException nếu có lỗi. Chỉ áp dụng cho ICommand (không chạy với Query).

LoggingBehavior

Ghi log [START] trước khi xử lý và [END] sau khi xử lý. Nếu request mất > 3 giây thì ghi thêm log [PERFORMANCE] để cảnh báo chậm.

3.3. Exception Handling

CustomExceptionHandler (IExceptionHandler) bắt tất cả exception và map sang HTTP response theo dạng ProblemDetails:

Exception Type	HTTP Status	Ghi chú
NotFoundException	404 Not Found	Entity không tồn tại
ValidationException	400 Bad Request	Dữ liệu đầu vào không hợp lệ
BadRequestException	400 Bad Request	Request không hợp lệ
InternalServerError	500 Internal Server Error	Lỗi server nội bộ
Exception (mặc định)	500 Internal Server Error	Các exception chưa xử lý

Response trả về theo RFC 7807 ProblemDetails có thêm 'traceId' để debug và 'ValidationErrors' nếu là ValidationException.

3.4. Messaging - Integration Events

```
// IntegrationEvent - base record cho mỗi integration event
public record IntegrationEvent
{
    public Guid Id => Guid.NewGuid();
    public DateTime OccurredOn => DateTime.Now;
    public string EventType => GetType().AssemblyQualifiedName;
}

// BasketCheckoutEvent - event được publish khi checkout giỏ hàng
public record BasketCheckoutEvent : IntegrationEvent
{
    public string UserName { get; set; }
    public Guid CustomerId { get; set; }
    public decimal TotalPrice { get; set; }
    // Shipping/Billing Address fields...
    // Payment fields (CardName, CardNumber, Expiration, CVV, PaymentMethod)
}
```

3.5. MassTransit Extensions

AddMessageBroker() là extension method dùng chung để cấu hình MassTransit + RabbitMQ. Được gọi trong Basket.API (publish event) và Ordering.API (consume event). Sử dụng kebab-case endpoint name formatter (basket-checkout-event).

4. CATALOG SERVICE - QUẢN LÝ SẢN PHẨM

Catalog.API là service quản lý danh mục sản phẩm. Sử dụng Vertical Slice Architecture: mỗi feature là một thư mục riêng chứa cả Endpoint lẫn Handler.

4.1. Cấu trúc

```
Catalog.API/  
+-- Data/  
|   +-- CatalogInitialData.cs # Seed data khi khởi động (Development)  
+-- Exceptions/  
|   +-- ProductNotFoundException.cs  
+-- Models/  
|   +-- Product.cs # Domain model (Id, Name, Category, Description,  
ImageFile, Price)  
+-- Products/  
    +-- CreateProduct/  
    |   +-- CreateProductEndpoint.cs # POST /products  
    |   +-- CreateProductHandler.cs # Command + Validator + Handler  
    +-- DeleteProduct/...  
    +-- GetProducts/...  
    +-- GetProductById/...  
    +-- GetProductByCategory/...  
    +-- UpdateProduct/...
```

4.2. Model sản phẩm

```
public class Product  
{  
    public Guid Id { get; set; }  
    public string Name { get; set; }  
    public List<string> Category { get; set; } // Multi-category support  
    public string Description { get; set; }  
    public string ImageFile { get; set; }  
    public decimal Price { get; set; }  
}
```

4.3. Luồng tạo sản phẩm (Vertical Slice)

Luồng xử lý tạo sản phẩm thể hiện rõ ràng pattern Vertical Slice:

1. HTTP POST /products -> Carter routing đến CreateProductEndpoint
2. Endpoint map request -> CreateProductCommand (qua Mapster.Adapt)
3. ISender.Send(command) -> MediatR pipeline
4. ValidationBehavior chạy CreateProductCommandValidator: Name, Category, ImageFile, Price > 0
5. LoggingBehavior ghi log [START]
6. CreateProductCommandHandler tạo Product entity, gọi session.Store(), SaveChangesAsync()

7. Marten lưu vào PostgreSQL, tự sinh Id
8. Trả về CreateProductResult(Id) -> map sang response -> HTTP 201 Created

Ghi chú: Catalog.API sử dụng Marten (document database trên PostgreSQL), không cần tạo tables hay migration thủ công. Marten tự quản lý schema.

4.4. API Endpoints

Method	Endpoint	Mô tả
GET	/products	Lấy danh sách có phân trang (pageNumber, pageSize)
GET	/products/{id}	Lấy chi tiết sản phẩm theo Id
GET	/products/category/{category}	Lấy sản phẩm theo danh mục
POST	/products	Tạo sản phẩm mới
PUT	/products	Cập nhật sản phẩm
DELETE	/products/{id}	Xóa sản phẩm
GET	/health	Health check (PostgreSQL ping)

5. BASKET SERVICE - GIỎ HÀNG

Basket.API quản lý giỏ hàng (ShoppingCart) theo UserName. Đây là service phức tạp nhất vì tích hợp nhiều công nghệ: PostgreSQL (Marten), Redis (cache), gRPC (lấy giảm giá từ Discount), RabbitMQ (publish CheckoutEvent).

5.1. Models


```

public class ShoppingCart
{
    public string UserName { get; set; }
    public List<ShoppingCartItem> Items { get; set; } = new();
    public decimal TotalPrice => Items.Sum(x => x.Price * x.Quantity); // computed
}

public class ShoppingCartItem
{
    public int Quantity { get; set; }
    public string Color { get; set; }
    public decimal Price { get; set; }
    public Guid ProductId { get; set; }
    public string ProductName { get; set; }
}

```

5.2. Repository Pattern với Decorator

Đây là ví dụ điển hình của Decorator Pattern. IBasketRepository có 2 implementation:

BasketRepository (concrete implementation):

Lưu trữ thực sự vào PostgreSQL thông qua Marten IDocumentSession. Cả 3 phương thức GetBasket, StoreBasket, DeleteBasket đều gọi database.

CachedBasketRepository (decorator):

Bọc thêm cache Redis trên BasketRepository gốc mà không thay đổi BasketRepository. Khi GetBasket: kiểm tra Redis trước, nếu có thì trả về luôn, nếu không thì gọi database rồi lưu cache. Khi StoreBasket/DeleteBasket: gọi repository gốc, đồng thời cập nhật/xóa cache tương ứng.

```

// Đăng ký trong Program.cs:
services.AddScoped<IBasketRepository, BasketRepository>();
services.Decorate<IBasketRepository, CachedBasketRepository>(); // Scrutor

// Kết quả: khi inject IBasketRepository, sẽ nhận CachedBasketRepository
// CachedBasketRepository bọc BasketRepository bên trong

```

5.3. Giảm giá qua gRPC

Khi StoreBasket, Basket.API gọi Discount.Grpc qua gRPC để lấy số tiền giảm giá cho mỗi sản phẩm. Kết nối được cấu hình với DangerousAcceptAnyServerCertificateValidator (chỉ dùng cho Development vì dùng self-signed certificate).

```
// Trong StoreBasketHandler
foreach (var item in basket.Items)
{
    var coupon = await discountProto.GetDiscountAsync(
        new GetDiscountRequest { ProductName = item.ProductName });
    item.Price -= coupon.Amount; // Trừ tiền giảm giá trực tiếp vào giá
}
```

5.4. Checkout và Publish Event

Khi người dùng checkout, Basket.API:

1. Lấy ShoppingCart từ repository (kiểm tra tồn tại)
2. Tạo BasketCheckoutEvent từ BasketCheckoutDto (Mapster.Adapt)
3. Set TotalPrice = basket.TotalPrice
4. Publish event lên RabbitMQ qua IPublishEndpoint (MassTransit)
5. Xóa giỏ hàng của user

Lưu ý: BasketCheckoutEvent chứa thông tin địa chỉ giao hàng, thanh toán. Ordering.API sẽ consume event này để tạo đơn hàng.

5.5. API Endpoints

Method	Endpoint	Mô tả
GET	/basket/{userName}	Lấy giỏ hàng theo UserName
POST	/basket	Cập nhật giỏ hàng (thêm/sửa sản phẩm)
DELETE	/basket/{userName}	Xóa giỏ hàng
POST	/basket/checkout	Checkout: publish event, xóa giỏ hàng
GET	/health	Health check (PostgreSQL + Redis)

6. DISCOUNT gRPC SERVICE - MÃ GIẢM GIÁ

Discount.Grpc là service đơn giản nhất trong dự án. Nó cung cấp giao diện gRPC để quản lý mã giảm giá (Coupon) lưu trong SQLite. Service được gọi nội bộ bởi Basket.API khi tính giá giỏ hàng.

6.1. Proto Contract

```
service DiscountProtoService {
    rpc GetDiscount (GetDiscountRequest) returns (CouponModel);
    rpc CreateDiscount (CreateDiscountRequest) returns (CouponModel);
    rpc UpdateDiscount (UpdateDiscountRequest) returns (CouponModel);
    rpc DeleteDiscount (DeleteDiscountRequest) returns (DeleteDiscountResponse);
}

message CouponModel {
    int32 id = 1;
    string productName = 2;
    string description = 3;
    int32 amount = 4;
}
```

6.2. Model và Database

```
public class Coupon
{
    public int Id { get; set; }
    public string ProductName { get; set; } // Key để tra cứu
    public string Description { get; set; }
    public int Amount { get; set; } // Số tiền giảm (không phải %)
}
```

Database: SQLite (file discountdb), quản lý bởi EF Core. Migration tự động áp dụng khi khởi động qua UseMigration() extension method.

6.3. GetDiscount Logic

Nếu không tìm thấy coupon theo productName, trả về coupon giá trị 0 ('No Discount') thay vì throw exception. Điều này đảm bảo Basket.API không bị lỗi khi sản phẩm không có giảm giá.

Ghi chú: Discount.Grpc chỉ được Basket.API gọi nội bộ qua Docker network. Không có route nào từ YARP Gateway đến service này.

7. ORDERING SERVICE - XỬ LÝ ĐƠN HÀNG

Ordering là service phức tạp nhất, được xây dựng theo Clean Architecture và Domain-Driven Design (DDD). Có 4 project: Ordering.Domain, Ordering.Application, Ordering.Infrastructure, Ordering.API.

7.1. Clean Architecture - 4 Layers

Layer	Nội dung	Phụ thuộc
Domain	Order, Customer, OrderItem, ValueObjects, DomainEvents	Không phụ thuộc ai
Application	Commands, Queries, Handlers, DTOs, IApplicationDbContext	Chỉ phụ thuộc Domain
Infrastructure	ApplicationDbContext, Interceptors, Migrations, EF Configurations	Phụ thuộc Application
API	Minimal API Endpoints, DI setup	Phụ thuộc cả 3 layer trên

7.2. Domain Layer - DDD Models

Aggregate và Entity:

```
public abstract class Aggregate<TId> : Entity<TId>, IAggregate<TId>
{
    private readonly List<IDomainEvent> _domainEvents = new();
    public IReadOnlyList<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();
    public void AddDomainEvent(IDomainEvent domainEvent) =>
        _domainEvents.Add(domainEvent);
    public IDomainEvent[] ClearDomainEvents() { ... }
}

// Order là Aggregate Root
public class Order : Aggregate<OrderId>
{
    private readonly List<OrderItem> _orderItems = new();
    public IReadOnlyList<OrderItem> OrderItems => _orderItems.AsReadOnly();

    public static Order Create(OrderId id, CustomerId customerId, ...)
    {
        var order = new Order { ... };
        order.AddDomainEvent(new OrderCreatedEvent(order)); // Raise Domain Event
        return order;
    }
}
```

Value Objects:

Value Objects là objects bất biến, không có identity, so sánh theo giá trị. Tất cả có static factory method Of() để validate:

Value Object	Trường dữ liệu	Validation
OrderId	Guid Value	Không được Guid.Empty
CustomerId	Guid Value	Không được Guid.Empty
OrderName	string Value	Không được null/empty
Address	FirstName, LastName, Email, AddressLine, Country, State, ZipCode	Email và AddressLine bắt buộc
Payment	CardName, CardNumber, Expiration, CVV, PaymentMethod	CVV <= 3 ký tự, CardNumber bắt buộc
ProductId	Guid Value	Không được Guid.Empty

7.3. Domain Events

```
// Raise khi tạo đơn hàng
public record OrderCreatedEvent(Order Order) : IDomainEvent;

// Raise khi cập nhật đơn hàng
public record OrderUpdatedEvent(Order Order) : IDomainEvent;

// Event handlers (trong Application layer)
OrderCreatedEventHandler -> ghi log 'Order created, start order fulfillment'
OrderUpdatedEventHandler -> ghi log 'Order updated'
```

7.4. Application Layer - CQRS Commands & Queries

Command/Query	Mô tả
CreateOrderCommand	Tạo đơn hàng mới từ OrderDto. Handler tạo Order aggregate, gọi context.Orders.Add()
UpdateOrderCommand	Cập nhật đơn hàng. Handler lấy Order, gọi order.Update()
DeleteOrderCommand	Xóa đơn hàng. Handler lấy Order, gọi context.Orders.Remove()
GetOrdersQuery	Lấy tất cả đơn hàng có phân trang và OrderExtensions.ToOrderDtoList()
GetOrdersByCustomerQuery	Lấy đơn hàng theo CustomerId
GetOrdersByNameQuery	Lấy đơn hàng theo OrderName (tìm kiếm)

7.5. Integration Event - BasketCheckoutEventHandler

Ordering.Application consume BasketCheckoutEvent từ RabbitMQ qua MassTransit IConsumer<BasketCheckoutEvent>. Khi nhận được event, handler tạo CreateOrderCommand và gửi qua MediatR ISender.

Lưu ý: Hiện tại, OrderItems trong BasketCheckoutEventHandler đang dùng giá trị hard-coded (2 items cố định). Đây là điểm cần cải tiến để xử lý đơn hàng thực tế.

7.6. Infrastructure Layer

ApplicationDbContext:

EF Core DbContext với SQL Server. Có DbSet<Order>, DbSet<Customer>, DbSet<Product>, DbSet<OrderItem>.

AuditableEntityInterceptor (SaveChangesInterceptor):

Tự động set CreatedBy, CreatedAt khi thêm mới; LastModifiedBy, LastModified khi sửa. Hiện tại hard-code 'mehmet'.

DispatchDomainEventsInterceptor (SaveChangesInterceptor):

Khi gọi SaveChanges, tự động lấy tất cả DomainEvents từ các Aggregate đang được track, rồi dispatch qua IMediator.Publish(). Đây là cơ chế tự động fire domain events sau khi lưu database.

8. YARP API GATEWAY

YARP (Yet Another Reverse Proxy) là API Gateway duy nhất cho tất cả client. Nó nhận request từ Shopping.Web và forward tới đúng service tương ứng dựa trên URL prefix.

8.1. Routing Configuration

```
// appsettings.json - ReverseProxy section

Routes:
  /catalog-service/{**catch-all} -> catalog-cluster (catalog.api:8080)
  /basket-service/{**catch-all}   -> basket-cluster  (basket.api:8080)
  /ordering-service/{**catch-all} -> ordering-cluster (ordering.api:8080)
    + RateLimiterPolicy: 'fixed'

// PathPattern transform: xóa prefix service khi forward
// /catalog-service/products/{id} -> /products/{id}
```

8.2. Rate Limiting

YARP được cấu hình với fixed window rate limiter áp dụng cho Ordering route. Đây là bảo vệ service khỏi bị gọi quá nhiều request trong thời gian ngắn.

8.3. Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));
builder.Services.AddRateLimiter(rateLimiterOptions => { ... });

var app = builder.Build();
app.UseRateLimiter();
app.MapReverseProxy();
app.Run();
```

9. SHOPPING.WEB - GIAO DIỆN NGƯỜI DÙNG

Shopping.Web là ứng dụng ASP.NET Razor Pages, đóng vai trò frontend của hệ thống. Nó giao tiếp với các microservices thông qua YARP Gateway bằng Refit HTTP client.

9.1. Refit HTTP Client Interfaces

```
[Get("/catalog-service/products?pageNumber={pageNumber}&pageSize={pageSize}")]
Task<GetProductsResponse> GetProducts(int? pageNumber = 1, int? pageSize = 10);

[Get("/catalog-service/products/{id}")]
Task<GetProductByIdResponse> GetProduct(Guid id);

[Get("/catalog-service/products/category/{category}")]
Task<GetProductByCategoryResponse> GetProductsByCategory(string category);

// IBasketService (GET, POST, DELETE basket; POST checkout)
// IOrderingService (GET orders by customer)
```

9.2. Razor Pages

Page	Chức năng
Index	Trang chủ, hiển thị danh sách sản phẩm
ProductList	Danh sách sản phẩm, lọc theo category
ProductDetail	Chi tiết sản phẩm, thêm vào giỏ hàng
Cart	Xem giỏ hàng, thay đổi số lượng
Checkout	Nhập thông tin giao hàng, thanh toán
Confirmation	Xác nhận đơn hàng sau checkout
OrderList	Lịch sử đơn hàng của khách hàng

Tất cả Refit services cấu hình base address là GatewayAddress (<http://yarpapigateway:8080>), đảm bảo tất cả request đi qua YARP.

10. CƠ SỞ HẠ TẦNG VÀ TRIỂN KHAI

10.1. Docker Compose

Toàn bộ hệ thống chạy trên Docker Compose. Mỗi service được định nghĩa trong `docker-compose.yml` (image + build) và `docker-compose.override.yml` (environment, ports, depends_on).

10.2. Database per Service

Service	Database	ORM/Tool	Migration
Catalog	PostgreSQL (CatalogDb)	Marten	Auto (Marten document)
Basket	PostgreSQL (BasketDb)	Marten	Auto (Marten document)
Discount	SQLite (discountdb)	EF Core	Auto UseMigration()
Ordering	SQL Server (OrderDb)	EF Core	Auto InitialiseDatabaseAsync()

10.3. Hướng dẫn chạy dự án

1. Cài đặt Docker Desktop (Windows/Mac) hoặc Docker Engine (Linux)
2. Clone repository
3. Di chuyển vào thư mục src:

```
cd src
```

4. Khởi động tất cả services:

```
docker-compose up -d
```

5. Kiểm tra trạng thái:

```
docker-compose ps
```

6. Truy cập ứng dụng:
 - Shopping.Web: `http://localhost:6005`
 - YARP Gateway: `http://localhost:6004`
 - Catalog API: `http://localhost:6000`
 - Basket API: `http://localhost:6001`
 - Discount gRPC: `http://localhost:6002`
 - Ordering API: `http://localhost:6003`

- RabbitMQ Management: <http://localhost:15672> (guest/guest)

10.4. Health Checks

Mỗi service (trừ Discount.Grpc) có /health endpoint kiểm tra kết nối database và cache. Kết quả trả về theo định dạng JSON chi tiết qua UIResponseWriter.

11. LUỒNG GIAO TIẾP GIỮA CÁC SERVICES

11.1. Luồng xem và thêm vào giỏ hàng

1. Người dùng truy cập Shopping.Web (port 6005)
2. Shopping.Web gọi ICatalogService.GetProducts() -> YARP Gateway
3. YARP forward /catalog-service/products -> Catalog.API
4. Catalog.API lấy danh sách sản phẩm từ PostgreSQL (Marten)
5. Hiển thị sản phẩm, người dùng click 'Add to Cart'
6. Shopping.Web gọi IBasketService.StoreBasket() -> YARP -> Basket.API
7. Basket.API gọi Discount.gRPC (nội bộ) lấy số tiền giảm giá
8. Lưu ShoppingCart vào PostgreSQL + cache vào Redis

11.2. Luồng Checkout (quan trọng nhất)

```

Shopping.Web
-> POST /basket-service/basket/checkout (qua YARP)
-> Basket.API CheckoutBasketHandler
  1. GetBasket(userName) <- PostgreSQL / Redis
  2. Map BasketCheckoutDto -> BasketCheckoutEvent
  3. Set eventMessage.TotalPrice = basket.TotalPrice
  4. publishEndpoint.Publish(BasketCheckoutEvent) -> RabbitMQ
  5. DeleteBasket(userName)
  6. Return IsSuccess: true

RabbitMQ (basket-checkout-event queue)
-> Ordering.API BasketCheckoutEventHandler (MassTransit IConsumer)
  1. Map event -> CreateOrderCommand
  2. sender.Send(CreateOrderCommand) -> MediatR
    -> CreateOrderHandler
      - Tạo Order aggregate (Order.Create())
      - Raise OrderCreatedEvent (nội bộ)
      - context.Orders.Add(order)
      - SaveChangesAsync()
        -> DispatchDomainEventsInterceptor
          -> mediator.Publish(OrderCreatedEvent)
          -> OrderCreatedEventHandler (ghi log)
        -> AuditableEntityInterceptor
          -> Set CreatedAt, LastModified tự động

```

11.3. Giao thức giao tiếp

Từ	Đến	Giao thức	Mô tả
Shopping.Web	YARP Gateway	HTTP/Refit	Tất cả request của Web UI
YARP Gateway	Catalog/Basket/Ordering	HTTP Reverse Proxy	Forward và transform URL
Basket.API	Discount.Grpc	gRPC (sync)	Lấy giảm giá khi thêm vào giỏ hàng
Basket.API	RabbitMQ	AMQP (async publish)	Publish BasketCheckoutEvent
RabbitMQ	Ordering.API	AMQP (async consume)	Consume BasketCheckoutEvent

12. CÁC KHÁI NIỆM KIẾN TRÚC CỐT LÕI

12.1. Microservices Architecture

Mỗi service là một 'mini-application' độc lập: có database riêng, có thể deploy riêng, có thể scale độc lập. Dự án có 4 business services: Catalog, Basket, Discount, Ordering.

12.2. Vertical Slice Architecture (VSA)

Catalog và Basket tổ chức code theo feature (vertical slice) thay vì theo tầng (horizontal layer). Mỗi feature có tất cả code liên quan: Endpoint, Command/Query, Handler, Validator, Response - trong cùng một thư mục.

12.3. CQRS với MediatR

Commands thay đổi trạng thái (Create, Update, Delete), Queries chỉ đọc dữ liệu. MediatR làm trung gian, kèm theo ValidationBehavior và LoggingBehavior trong pipeline.

12.4. Domain-Driven Design (Ordering)

Order là Aggregate Root có private List<OrderItem>, enforce invariants (giá trị âm). Value Objects (OrderId, Address, Payment) là bất biến, có static factory method validation. Domain Events (OrderCreatedEvent, OrderUpdatedEvent) được raise khi có thay đổi nghiệp vụ.

12.5. Clean Architecture (Ordering)

Domain layer không biết gì về Infrastructure. Application layer chỉ biết Domain và interfaces. Infrastructure cài đặt các interface. Dependency flow: API -> Application -> Domain (một chiều).

12.6. Decorator Pattern (Cache-Aside)

CachedBasketRepository decorator thêm Redis cache lên BasketRepository mà không thay đổi code gốc. Scrutor library cho phép services.Decorate<IBasketRepository, CachedBasketRepository>() tự động.

12.7. Event-Driven Architecture

BasketCheckoutEvent là Integration Event đi qua RabbitMQ, tách biệt Basket và Ordering. Basket không cần biết Ordering có tồn tại hay không.

12.8. API Gateway Pattern

YARP là single entry point cho Shopping.Web. Nó ẩn đi địa chỉ nội bộ của các services, thêm rate limiting, và có thể thêm authentication/SSL termination sau này.

13. API ENDPOINTS THAM KHẢO

Tất cả endpoints trên được truy cập qua YARP Gateway (<http://localhost:6004>) với prefix phù hợp:

13.1. Catalog Service (/catalog-service/...)

Method	Path	Request Body	Response
GET	/products?pageNumber=1&pageSize=10	-	PaginatedResult<Product>
GET	/products/{id}	-	Product
GET	/products/category/{category}	-	List<Product>
POST	/products	{Name, Category, Description, ImageFile, Price}	{ Id } 201
PUT	/products	{Id, Name, Category, Description, ImageFile, Price}	{ IsSuccess }
DELETE	/products/{id}	-	{ IsSuccess }

13.2. Basket Service (/basket-service/...)

Method	Path	Request Body	Response
GET	/basket/{userName}	-	ShoppingCart
POST	/basket	ShoppingCart	ShoppingCart
DELETE	/basket/{userName}	-	{ IsSuccess }
POST	/basket/checkout	BasketCheckoutDto	{ IsSuccess }

13.3. Ordering Service (/ordering-service/...)

Method	Path	Request Body	Response
GET	/orders?pageNumber=1&pageSize=10	-	PaginatedResult<OrderDto>
GET	/orders/customer/{customerId}	-	List<OrderDto>
GET	/orders/{orderId}	-	List<OrderDto>
POST	/orders	OrderDto	{ OrderId }

Method	Path	Request Body	Response
PUT	/orders	OrderDto	{ IsSuccess }
DELETE	/orders/{id}	-	{ IsSuccess }

14. YÊU CẦU ĐỐI VỚI PROJECT MICROSERVICES

Sau khi đã hoàn thành project mẫu EShopMicroservices, sinh viên cần thực hiện thêm các yêu cầu sau:

14.1. Chạy lại toàn bộ hệ thống

Sinh viên phải chạy lại project bằng Docker Compose hoặc cách triển khai tương đương, đảm bảo các service hoạt động đầy đủ, bao gồm:

- Catalog Service
- Basket Service
- Discount gRPC Service
- Ordering Service
- YARP API Gateway
- Shopping.Web
- PostgreSQL / SQL Server / Redis / RabbitMQ

Sinh viên cần chụp hình minh chứng cho các nội dung sau:

- Kết quả docker-compose ps hoặc danh sách container đang chạy.
- Giao diện Shopping.Web truy cập thành công.
- Danh sách sản phẩm từ Catalog Service.
- Thao tác thêm sản phẩm vào giỏ hàng.
- Thao tác checkout thành công.
- RabbitMQ Management UI thể hiện message/event được publish hoặc consume.
- Dữ liệu đơn hàng được tạo trong Ordering Service.

14.2. Phát triển thêm 1–2 chức năng mới

Sinh viên cần mở rộng project bằng cách phát triển thêm ít nhất **01 chức năng mới**, khuyến khích **02 chức năng**, có liên quan đến kiến trúc Microservices hiện tại.

Một số gợi ý chức năng:

- Thêm chức năng quản lý danh mục sản phẩm.
- Thêm chức năng tìm kiếm sản phẩm theo tên hoặc khoảng giá.
- Thêm chức năng cập nhật số lượng sản phẩm trong giỏ hàng.
- Thêm chức năng xem chi tiết đơn hàng.
- Thêm chức năng hủy đơn hàng.
- Thêm chức năng trạng thái đơn hàng: Pending, Paid, Shipped, Cancelled.

- Thêm chức năng gửi event mới qua RabbitMQ, ví dụ OrderCancelledEvent hoặc OrderStatusChangedEvent.
- Thêm chức năng áp dụng mã giảm giá theo phần trăm thay vì số tiền cố định.
- Thêm chức năng logging/correlation ID để theo dõi request qua nhiều service.

14.3. Yêu cầu kỹ thuật đối với chức năng mới

Chức năng mới cần thể hiện được việc sinh viên hiểu kiến trúc của project, cụ thể:

- Có endpoint API phù hợp.
- Có Command/Query/Handler nếu service đang dùng CQRS.
- Có validation dữ liệu đầu vào nếu cần.
- Có cập nhật database tương ứng.
- Nếu liên quan nhiều service, cần dùng HTTP/gRPC hoặc RabbitMQ hợp lý.
- Nếu có UI, cần cập nhật Shopping.Web để người dùng thao tác được.
- Có chụp hình minh chứng kết quả trước và sau khi chạy chức năng.

14.4. Sản phẩm cần nộp

Sinh viên cần nộp:

- Source code project đã cập nhật.
- File báo cáo ngắn mô tả chức năng đã phát triển.
- Hình chụp minh chứng hệ thống chạy thành công.
- Hình chụp minh chứng chức năng mới hoạt động.
- Mô tả ngắn luồng xử lý của chức năng mới.
- Nếu có sử dụng RabbitMQ/gRPC/API Gateway, cần giải thích rõ vai trò của thành phần đó trong chức năng mới.

14.5. Mức đánh giá dự kiến

- Chạy được hệ thống gốc và có minh chứng đầy đủ: 30%.
- Phát triển chức năng mới đúng yêu cầu: 40%.
- Áp dụng đúng kiến trúc Microservices/CQRS/DDD/Event-driven nếu có: 20%.
- Báo cáo rõ ràng, hình ảnh minh chứng đầy đủ: 10%.